

✓ Пункт 1. OCR-распознавание текста из PDF

✓ Установка библиотек

```
!apt-get update -qq
!apt-get install -y tesseract-ocr tesseract-ocr-rus poppler-utils -qq

!pip install pytesseract pdf2image pillow pandas -q
```

```
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease
W: Failed to fetch https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu/dists/jammy/InRelease Could not connect to ppa.launch
W: Failed to fetch https://ppa.launchpadcontent.net/ubuntuugis/ppa/ubuntu/dists/jammy/InRelease Unable to connect to ppa.launchp
W: Some index files failed to download. They have been ignored, or old ones used instead.
Selecting previously unselected package poppler-utils.
(Reading database ... 118194 files and directories currently installed.)
Preparing to unpack .../poppler-utils_22.02.0-2ubuntu0.12_amd64.deb ...
Unpacking poppler-utils (22.02.0-2ubuntu0.12) ...
Selecting previously unselected package tesseract-ocr-rus.
Preparing to unpack .../tesseract-ocr-rus_1%3a4.00~git30-7274cfa-1.1_all.deb ...
Unpacking tesseract-ocr-rus (1:4.00~git30-7274cfa-1.1) ...
Setting up poppler-utils (22.02.0-2ubuntu0.12) ...
Setting up tesseract-ocr-rus (1:4.00~git30-7274cfa-1.1) ...
Processing triggers for man-db (2.10.2-1) ...
```

```
import pytesseract
from pdf2image import convert_from_path
from google.colab import files
import pandas as pd
import re
import os
```

✓ Загрузка файла

```
uploaded = files.upload()

pdf_path = list(uploaded.keys())[0]
print("Загруженный файл:", pdf_path)
```

НИР втор...равок.pdf
НИР второй этап _ версия после правок.pdf(application/pdf) - 863770 bytes, last modified: 09.11.2025 - 100% done
 Saving НИР второй этап _ версия после правок.pdf to НИР второй этап _ версия после правок.pdf
 Загруженный файл: НИР второй этап _ версия после правок.pdf

✓ Конвертация в изображение

```
pages = convert_from_path(pdf_path, dpi=300)

print(f"Количество страниц в PDF: {len(pages)}")

for i, page in enumerate(pages):
    image_name = f"page_{i+1}.png"
    page.save(image_name, "PNG")
    print(f"Сохранено изображение: {image_name}")
```

```
Количество страниц в PDF: 12
Сохранено изображение: page_1.png
Сохранено изображение: page_2.png
Сохранено изображение: page_3.png
Сохранено изображение: page_4.png
Сохранено изображение: page_5.png
Сохранено изображение: page_6.png
Сохранено изображение: page_7.png
Сохранено изображение: page_8.png
Сохранено изображение: page_9.png
Сохранено изображение: page_10.png
Сохранено изображение: page_11.png
Сохранено изображение: page_12.png
```

Распознавание текста

```
recognized_texts = []

for i, page in enumerate(pages):
    text = pytesseract.image_to_string(page, lang="rus+eng")
    recognized_texts.append(text)

    print(f"\n--- Страница {i+1} ---\n")
    print(text[:1000])
```

В ходе анализа и реализации алгоритма СКАРПА был получен способ реконструкции параллельного сканирования. Так как на данный момент нет аппарата, на котором можно проверить точность этого алгоритма, то тестирование производилось на предоставленных искусственно созданных

данных (из полного K-пространства).

Реализовывать этот алгоритм помогли: Ваганов Ярослав Сергеевич, ИСУ 367123 — поиск источников и данных для тестирования Py^{оп};

Тымковский Константин Максимович, ИСУ 368935 — внедрение в Ш1\$.то.ги.
11 Источники

1. Progress in Nuclear Magnetic Resonance Spectroscopy // smallpdf
URL: <https://smallpdf.com/ru/file#s=6e90a8e1-b413-4af5-b398-6516b2bal> lcd (gata oOamenua: 21.01.2025).

2. pygrappa // github URL: <https://github.com/mckib2/pygrappa> (mata обращения: 21.01.2025).

3. numpy // github URL: <https://github.com/numpy/numpy> (дата обращения: 21.01.2025).

4. matplotlib // github URL: <https://github.com/matplotlib/matplotlib> (дата обращения: 21.01.2025).

5.

--- Страница 12 ---

7. Nana R., Hao T., Heberlein K., LaConte S.M., Hu X. Cross-validation-based kernel support selection for improved GRAPPA reconstruction.

Magnetic Resonance in Medicine 59:819–825 (2008).

8. David J Larkman D.J., Nunes R.G. Parallel magnetic resonance imaging. Phys. Med. Biol. 52 (2007) B15–B55.

9. Griswold M.A., Jakob P.M., Heidemann R.M., Nittka M., Jellus V., Wang J., Kiefer B., Haase A. Generalized autocalibrating partially parallel acquisitions (GRAPPA). Magn Reson Med 2003; 47: 1202–1210.

10. Wilson D.L., Huo D. Robust GRAPPA. United States Patent No.: US 8,222,900 B2 Jul. 17, 2012.

11. Wang Z., Wang J., Detre J.A. Improved data reconstruction method for GRAPPA. Magnetic Resonance in Medicine 54:738–742 (2005).

12.000 «ДЖЭТ-ЛАБ». Описание метода реконструкции ЗЕМЬЕ.

13. Wang H., Liang D., King K.F., Nagarsekar G., Ying L. Cross-sampled GRAPPA for parallel MRI. Department of Electrical Engineering and Computer Science, University of Wisconsin, Milwaukee WI 53201 USA.

14. De

Объединение текста со всех страниц

```
raw_text = "\n\n".join(recognized_texts)

with open("ocr_raw_text.txt", "w", encoding="utf-8") as f:
    f.write(raw_text)

print("Исходный OCR-текст сохранён в файл ocr_raw_text.txt")
```

Исходный OCR-текст сохранён в файл `ocr_raw_text.txt`

- Очистка текста

```
def clean_ocr_text(text):
    # Убираем лишние пробелы
    text = re.sub(r"[\t]+", " ", text)

    # Убираем слишком большое количество переносов строк
    text = re.sub(r"\n{3,}", "\n\n", text)

    # Убираем пробелы перед знаками препинания
    text = re.sub(r"\s+([.,!?:;])", r"\1", text)

    # Убираем лишние пробелы в начале и конце строк
    lines = [line.strip() for line in text.split("\n")]
    text = "\n".join(lines)

    return text.strip()

clean_text = clean_ocr_text(raw_text)

with open("ocr_clean_text.txt", "w", encoding="utf-8") as f:
    f.write(clean_text)

print(clean_text[:1500])
```

СОДЕРЖАНИЕ

[illegible]

- ▼ Скачивание результата

```
files.download("ocr_raw_text.txt")
files.download("ocr_clean_text.txt")
```

- ✓ Пункт 2. Постобработка и очистка распознанного текста

- Чтение сырого текста и очистка

```
with open("ocr_raw_text.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

print(raw_text[:2000])
```

СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ..... линии ини иинииниининии 2

1 Описание объекта исследования..... линии ини иийининьнининия 2

1.1 Предметная область..... линии ини ини ийининининияния 3

2 Постановка задачи нии ининининени 3

2.1 Цели и задачи..... ини ненененьниная 3

3 ВІВІВОІІ ОО АКТУАЛІВНОСТН.....с.cccccccsssscceesessececacesseeeeeeseseseeeeasesaes 4
ТЕОРЕТИЧЕСКОЕ ИССЛЕДОВАНИЕ..... ини иийиининине 5

4 Цель и задачи теоретического исследования..... инъ иининины 5

4.1 Цель..... нии ненененьнины 5

4.2 Задачи нии нининенениния 5

5 Результаты теоретического исследования и ининининининья 5

6 Выводы о применимости результатов..... ини иийиинининина 7

ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ.....иийинининининни 8

7 Возможность использования алгоритма в низком поле..... 9

8 Обучающая реализация алгоритма линии иийининийиинининье 9

9 Реализация алгоритма как функция сервиса реконструкции

томографа 1,5 Тл..... линии нии нинининининье 10

ЗАКЛЮЧЕНИЕ сеcccscscscscsecescscscscscscscusecctuaeecceceseeaaueeeyees 11

10 ВВІВІОВОІІ .і.0.....ссессеесеесеееcccscscscscscscscscscscscascscscensecccsesecesceaeaessenssecs 11

1] Источники..... ии иинньиньн 11

ПОСТАНОВКА ЗАДАЧИ

1 Описание объекта исследования

Процесс исследования МРТ очень продолжительный во времени, потому что для получения изображения необходимо собрать большое количество данных частотного кодирования. Для получения этих данных производят многократные возбуждающие импульсы, которые направляют спин в нужной для исследования ориентации. Для возвращения состояния в исходное положение так же нужно время. Такие импульсы собирают в последовательности и многократно повторяют, получается набор данных, из которых и строится К-пространство (частотное пространство изображения), а затем МРТ изображение.

90x 180y 180y ,---, 180y 180y 180:
| |

```
import re

def basic_clean_text(text):
    # Замена неразрывных пробелов на обычные
    text = text.replace("\xa0", " ")

    # Удаление лишних пробелов и табуляций
    text = re.sub(r"[ \t]+", " ", text)

    # Удаление пробелов перед знаками препинания
    text = re.sub(r"\s+([,;!?:])", r"\1", text)

    # Удаление лишних переносов строк
    text = re.sub(r"\n{3,}", "\n\n", text)

    # Очистка пробелов в начале и конце каждой строки
    lines = [line.strip() for line in text.split("\n")]

    return "\n".join(lines).strip()

clean_text_stage_1 = basic_clean_text(raw_text)

print(clean_text_stage_1[:2000])
```


СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ..... ини иинининининни 2

1 Описание объекта исследования..... линии ини иииининьнинининия 2

1.1 Предметная область..... линии ини ини иинининининининия 3

2 Постановка задачи..... нии иинининенни 3

2.1 Цели и задачи..... ии ненененьнина 3

```
3 BbIBOJII 00 AKTYAIIBHOCTH.....c.cccccccssssccceceesessecececessesseeeeeeseseseeeeesesae 4
```

ТЕОРЕТИЧЕСКОЕ ИССЛЕДОВАНИЕ..... ини иииииининине 5

4 Цель и задачи теоретического исследования..... инь иниинини 5

4.1 Цель..... нии ненененьнини 5

4.2 Задачи..... нии нининененининия 5

5 Результаты теоретического исследования..... и иининининининьнииея 5

ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ.....иининининининни 8

7 Возможность использования алгоритма в низком поле..... 9

8 Обучающая реализация алгоритма..... линии ииининининининининьне 9

9 Реализация алгоритма как функция сервиса реконструкции

томографа 1,5 Тл..... линии нии нинининининине 10

ЗАКЛЮЧЕНИЕ..... сеесссссссссссеессссссссссуеессуаееесеесеееееееееее 11

```
10 BBIB00XDBI..0.....ccecceeccecccseccsscccecccescececccasccccsscensecccesceesesceaeaaaessensecs 11
```

1] Источники_..... ии ии ннинньнинь 11

ПОСТАНОВКА ЗАДАЧИ

1 Описание объекта исследования

Процесс исследования МРТ очень продолжительный во времени, потому что для получения изображения необходимо собрать большое количество данных частотного кодирования. Для получения этих данных производят многократные возбуждающие импульсы, которые направляют спин в нужной для исследования ориентации. Для возвращения состояния в исходное положение так же нужно время. Такие импульсы собирают в последовательности и многократно повторяют, получается набор данных, из которых и строится К-пространство (частотное пространство изображения), а затем МРТ изображение.

90x 180y 180y ,---, 180y 180y 180:

| | | sign

```
def remove_noise_lines(text):
    clean_lines = []

    for line in text.split("\n"):
        line = line.strip()

        # Пропускаем пустые строки
        if line == "":
            clean_lines.append(line)
            continue

        # Удаляем слишком короткие строки из 1 символа
        if len(line) == 1:
            continue

        # Удаляем строки, состоящие только из спецсимволов
        if re.fullmatch(r"[\W_]+", line):
            continue

        clean_lines.append(line)

    return "\n".join(clean_lines).strip()

clean_text_stage_2 = remove_noise_lines(clean_text_stage_1)

print(clean_text_stage_2[:2000])
```

СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ..... ини иинининининни 2

1 Описание объекта исследования..... линии или ииинининьнинининия 2

1.1 Предметная область..... линии ини ини иинининининининия 3

ПОСТАНОВКА ЗАДАЧИ

1 Описание объекта исследования

Процесс исследования МРТ очень продолжительный во времени, потому что для получения изображения необходимо собрать большое количество данных частотного кодирования. Для получения этих данных производят многократные возбуждающие импульсы, которые направляют спин в нужной для исследования ориентации. Для возвращения состояния в исходное положение так же нужно время. Такие импульсы собирают в последовательности и многократно повторяют, получается набор данных, из которых и строится К-пространство (частотное пространство изображения), а затем МРТ изображение.

```
90x 180y 180y ,---, 180y 180y 180:
| | | sign
```

- ✦ Исправление частых ошибок

```
ocr_replacements = {
    "OCR": "OCR",
    "OCR": "OCR",
    "NER": "NER",
    "F1": "F1",
    "PDF": "PDF",
    "Text": "Text",
}

def fix_common_ocr_errors(text, replacements):
    for wrong, correct in replacements.items():
        text = text.replace(wrong, correct)
    return text

clean_text_stage_3 = fix_common_ocr_errors(clean_text_stage_2, ocr_replacements)

print(clean_text_stage_3[:2000])
```

СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ..... ини ининининини 2
1 Описание объекта исследования..... линии ини иииининьнинининия 2

1.1 Предметная область..... линии ини ини иинининининания 3
2 Постановка задачи..... нии ининининенни 3

2.1 Цели и задачи..... ини ненененьниная 3
3 ВВІВОДІІ ОО АКТУАЛІВНОСТН.....с.сссссссссссссссеесесеcececессеessееееееeseseееееesesae 4
ТЕОРЕТИЧЕСКОЕ ИССЛЕДОВАНИЕ..... ини иииинининине 5
4 Цель и задачи теоретического исследования..... инь иининини 5

4.1 Цель..... нии ненененьнини 5

4.2 Задачи..... нии нининененинния 5
5 Результаты теоретического исследования..... и инининининининья 5
6 Выводы о применимости результатов..... ини иииининининия 7
ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ.....ииинининининни 8

```
90x 180y 180y ,---, 180y 180y 180:
| | | sign
```

1.1 Предметная область..... линии инии иинининининияния З
2 Постановка задачи..... нии иининининения З

2.1 Цели и задачи..... инии ненененьнинияя З

3 ВВІВОДІІ ОО АКТУАЛІВНОСТН.....с.сссссссссссссссеесесеcececcesessseeeeeeseseseeeeeeesaes 4
ТЕОРЕТИЧЕСКОЕ ИССЛЕДОВАНИЕ..... инии ииининининине 5

4 Цель и задачи теоретического исследования..... инь иининини 5

4.1 Цель..... нии ненененьнини 5

4.2 Задачи..... нии нининененинниияя 5

5 Результаты теоретического исследования..... и иинининининининьяния 5

6 Выводы о применимости результатов..... инии ииинининининенея 7
ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ.....ииининининининни 8

7 Возможность использования алгоритма в низком поле..... 9

8 Обучающая реализация алгоритма..... линии ииининииинииинининье 9

9 Реализация алгоритма как

```
comparison_data = {
    "Этап": [
        "Исходный OCR-текст",
        "После базовой очистки",
        "После удаления мусорных строк",
        "После исправления частых ошибок"
    ],
    "Описание": [
        "Текст, полученный напрямую после OCR-распознавания",
        "Удалены лишние пробелы, табуляции и лишние переносы строк",
        "Удалены короткие и служебные строки, не несущие смысловой нагрузки",
        "Исправлены повторяющиеся ошибки распознавания"
    ],
    "Длина текста": [
        len(raw_text),
        len(clean_text_stage_1),
        len(clean_text_stage_2),
        len(clean_text_stage_3)
    ]
}

df_cleaning = pd.DataFrame(comparison_data)
df_cleaning
```

	Этап	Описание	Длина текста
0	Исходный OCR-текст	Текст, полученный напрямую после OCR-распознав...	13532
1	После базовой очистки	Удалены лишние пробелы, табуляции и лишние пер...	13441
2	После удаления мусорных строк	Удалены короткие и служебные строки, не несущи...	13427
3	После исправления частых ошибок	Исправлены повторяющиеся ошибки распознавания	13427

Далее: [New interactive sheet](#)

- ✓ Пункт 3. Оценка качества OCR

Цель: проверить, насколько хорошо OCR-движок распознал текст, и показать примеры ошибок.

Загрузка очищенного текста и разделение его на строки для удобства проверки.

```
with open("ocr_clean_text.txt", "r", encoding="utf-8") as f:
    clean_text = f.read()

print(clean_text[:3000])
```

```

90x 180y 180y ,---, 180y 180y 180:
| | | signal signal signal | signal signal
BP +++ oH ++ № 4+ oe
Dx

```

Dx

ox tnt trl deel dena

Dy | Dy
G A и | lh | 4
' "и "и PE PE y
02 Dz!

Gz ttt ttt th

aot Sty Navigator echoes

Рисунок 1 — Методика получения МРТ изображения, набор возбуждающих импульсов. Для уменьшения длительности исследования применяются методы параллельного сканирования (parallel imaging), в основе которых идет сбор не полной информации при использовании нескольких приемных катушек, информация с которых коррелирует и пространственные чувствительности которых различны. При этом недостающие данные «дособираются»

искусственным способом – с помощью алгоритмов.

Рисунок 2 – Пара: К-пространство и МРТ изображение.

1.1 Предметная область

Одним из алгоритмов параллельного сканирования является СКАРПА. Алгоритм использует данные из автокалибровки (АСЗ, прескан) для вычисления весовых коэффициентов приемных катушек. По этим весам и

дописывается недостающая информация со сканирования.

2 Постановка задачи

Современные

```
lines = clean_text.split("\n")
```

```
for i, line in enumerate(lines[:50], start=1):
    print(f"{i}: {line}")
```

1: СОДЕРЖАНИЕ
2:
3: ПОСТАНОВКА ЗАДАЧИ..... ини инининининни 2
4: 1 Описание объекта исследования..... линии ини ииииниьнинининния 2
5:
6: 1.1 Предметная область..... линии ини ини иииинининияания 3
7: 2 Постановка задачи..... нии ининининенни 3
8:
9: 2.1 Цели и задачи..... ини ненененьнина 3
10: 3 ВЪВОДИИ ОО АКТУАІВНОСТН.....с.ccccccsessssccccceeseccecceessseseeeeeseseseeeeasesaes 4
11: ТЕОРЕТИЧЕСКОЕ ИССЛЕДОВАНИЕ..... инии иииинининине 5
12: 4 Цель и задачи теоретического исследования..... инъ иининини 5
13:
14: 4.1 Цель..... нии ненененьнини 5
15:
16: 4.2 Задачи..... нии нининененинния 5
17: 5 Результаты теоретического исследования..... и инининининининьяиея 5
18: 6 Выводы о применимости результатов..... ини иииииинининина 7
19: ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ.....ииинининининни 8
20: 7 Возможность использования алгоритма в низком поле..... 9
21: 8 Обучающая реализация алгоритма..... линии иииининиийииинининье 9
22:
23: 9 Реализация алгоритма как функция сервиса реконструкции
24:
25: томографа 1,5 Тл..... линии нии нининининининье 10
26: ЗАКЛЮЧЕНИЕ..... сеессссссссссссеесссссссссссусеессуееесесеесееауееееееее 11
27: 10 BBIBOOXDBI..0.....cсecccсеесссеессссссссссссеессеесссассссссенсеессссеессеесаеаеаессенсеcs 11
28:

Page No. _____

Date ____/____/____

Загрузка эталонного текста

3.1/3.1 MB 60.4 MB/s eta 0:00:00

10/45

```
print("CER:", char_error_rate)
```

```
WER: 0.004617414248021108
CER: 0.0010415891674726582
```

```
ocr_metrics = pd.DataFrame({
    "Метрика": ["WER", "CER"],
    "Описание": [
        "Доля ошибок на уровне слов",
        "Доля ошибок на уровне символов"
    ],
    "Значение": [
        word_error_rate,
        char_error_rate
    ]
})

ocr_metrics
```

	Метрика	Описание	Значение	
0	WER	Доля ошибок на уровне слов	0.004617	
1	CER	Доля ошибок на уровне символов	0.001042	

Далее: [New interactive sheet](#)

Ручная проверка выявила такое:

```
import pandas as pd

ocr_errors = [
    {
        "Фрагмент OCR": "OCR",
        "Правильный вариант": "OCR",
        "Тип ошибки": "Смещение кириллической и латинской буквы",
        "Комментарий": "OCR перепутал латинскую О и кириллическую О"
    },
    {
        "Фрагмент OCR": "NER",
        "Правильный вариант": "NER",
        "Тип ошибки": "Смещение кириллической и латинской буквы",
        "Комментарий": "В слове использована кириллическая Е вместо латинской Е"
    },
    {
        "Фрагмент OCR": "F1",
        "Правильный вариант": "F1",
        "Тип ошибки": "Неверное распознавание символа",
        "Комментарий": "Буква F была распознана как похожий символ"
    }
]

df_ocr_errors = pd.DataFrame(ocr_errors)
df_ocr_errors
```

	Фрагмент OCR	Правильный вариант	Тип ошибки	Комментарий	
0	OCR	OCR	Смещение кириллической и латинской буквы	OCR перепутал латинскую О и кириллическую О	
1	NER	NER	Смещение кириллической и латинской буквы	В слове использована кириллическая Е вместо латинской Е	

Далее: [New interactive sheet](#)

✦ Пункт 4. Извлечение именованных сущностей с помощью готовых NER-моделей

Цель: применить готовые модели NER к очищенному тексту и выделить сущности: людей, места, организации, даты и произведения.

Будем сравнивать две модели:

SpaCy — простая и удобная модель для NER. Stanza — модель от Stanford NLP, хорошо работает с русским языком.

```
!pip install spacy stanza pandas -q
!python -m spacy download ru_core_news_sm
```

```

===== 1.7/1.7 MB 47.3 MB/s eta 0:00:00
===== 773.7/773.7 kB 38.5 MB/s eta 0:00:00
===== 608.4/608.4 kB 35.0 MB/s eta 0:00:00
===== 418.7/418.7 kB 25.9 MB/s eta 0:00:00

Collecting ru-core-news-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/ru\_core\_news\_sm-3.8.0/ru\_core\_news\_sm-3.8.0-py3-none-any.whl
    ===== 15.3/15.3 MB 23.2 MB/s eta 0:00:00
Collecting pymorphy3>=1.0.0 (from ru-core-news-sm==3.8.0)
  Downloading pymorphy3-2.0.6-py3-none-any.whl.metadata (2.4 kB)
Collecting dawg2-python>=0.8.0 (from pymorphy3>=1.0.0->ru-core-news-sm==3.8.0)
  Downloading dawg2_python-0.9.0-py3-none-any.whl.metadata (7.5 kB)
Collecting pymorphy3-dicts-ru (from pymorphy3>=1.0.0->ru-core-news-sm==3.8.0)
  Downloading pymorphy3_dicts_ru-2.4.417150.4580142-py2.py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: setuptools>=68.2.2 in /usr/local/lib/python3.12/dist-packages (from pymorphy3>=1.0.0->ru-core-news-sm==3.8.0)
  Downloading pymorphy3-2.0.6-py3-none-any.whl (53 kB)
    ===== 53.9/53.9 kB 3.1 MB/s eta 0:00:00
  Downloading dawg2_python-0.9.0-py3-none-any.whl (9.3 kB)
  Downloading pymorphy3_dicts_ru-2.4.417150.4580142-py2.py3-none-any.whl (8.4 MB)
    ===== 8.4/8.4 MB 61.8 MB/s eta 0:00:00

Installing collected packages: pymorphy3-dicts-ru, dawg2-python, pymorphy3, ru-core-news-sm
Successfully installed dawg2-python-0.9.0 pymorphy3-2.0.6 pymorphy3-dicts-ru-2.4.417150.4580142 ru-core-news-sm-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('ru_core_news_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.
```

```
import spacy
import stanza
import pandas as pd
```

```
with open("ocr_clean_text.txt", "r", encoding="utf-8") as f:
    text = f.read()
```

▼ Модель 1. SpaCy

```
nlp_spacy = spacy.load("ru_core_news_sm")
```

Извлечение сущностей

```
doc_spacy = nlp_spacy(text)

spacy_entities = []

for ent in doc_spacy.ents:
    spacy_entities.append({
        "Модель": "SpaCy",
        "Текст сущности": ent.text,
        "Тип сущности": ent.label_,
        "Начало": ent.start_char,
        "Конец": ent.end_char
    })

df_spacy = pd.DataFrame(spacy_entities)
df_spacy.head(20)
```


	Модель	Текст сущности	Тип сущности	Начало	Конец
0	SpaCy	Предметная область	LOC	145	163
1	SpaCy	BP +++ оН ++ № 4+ ое\nDx\n\nDx	ORG	2033	2060
2	SpaCy	РЕ	ORG	2119	2121
3	SpaCy	Предметная область	LOC	2698	2716
4	SpaCy	Цель	PER	3437	3441
5	SpaCy	GRAPPA	ORG	3487	3493
6	SpaCy	СКАРРА	ORG	3723	3729
7	SpaCy	К-	LOC	4108	4110
8	SpaCy	МРА	ORG	4198	4201
9	SpaCy	GRAPPA	ORG	4514	4520
10	SpaCy	К-	LOC	4760	4762
11	SpaCy	Алгоритм	ORG	4947	4955
12	SpaCy	ОКАРРА	ORG	4956	4962
13	SpaCy	GRAPPA	ORG	5182	5188
14	SpaCy	К-	LOC	6009	6011
15	SpaCy	СКАРРА	ORG	6507	6513
16	SpaCy	СВАРРА	ORG	7168	7174
17	SpaCy	Frequency Encoding (ky)	ORG	7492	7515
18	SpaCy	Ampo	ORG	7518	7522
19	SpaCy	К-	LOC	8688	8690

Далее: [New interactive sheet](#)

Количество сущностей

```
df_spacy["Тип сущности"].value_counts()
```

count	
Тип сущности	
ORG	50
LOC	10
PER	8

dtype: int64

▼ Модель 2. Stanza

```
stanza.download("ru")
nlp_stanza = stanza.Pipeline(
    lang="ru",
    processors="tokenize,ner"
)
```

```

Downloading https://raw.githubusercontent.com/stanfordnlp/stanza-
resources/main/resources_1.11.0.json: 439k/? [00:00<00:00, 11.5MB/s]
INFO:stanza:Downloaded file to /root/.cache/stanza/1.11.0/resources/resources.json
INFO:stanza:Downloading default packages for language: ru (Russian) ...

Downloading https://huggingface.co/stanfordnlp/stanza- 629M/629M [00:07<00:00, 80.0MB/s]
ru/resolve/v1.11.0/models/default.zip: 100%
INFO:stanza:Downloaded file to /root/.cache/stanza/1.11.0/resources/ru/default.zip
INFO:stanza:Finished downloading models and saved to /root/.cache/stanza/1.11.0/resources
INFO:stanza:Checking for updates to resources.json in case models have been updated. Note: this behavior can be turned off with
Downloading https://raw.githubusercontent.com/stanfordnlp/stanza- 439k/? [00:00<00:00, 26.1MB/s]
resources/main/resources_1.11.0.json:
INFO:stanza:Downloaded file to /root/.cache/stanza/1.11.0/resources/resources.json
INFO:stanza:Loading these models for language: ru (Russian):
=====
| Processor | Package |
|-----|-----|
| tokenize | syntagrus |
| ner      | wikiner   |
=====

```

Извлечение сущностей

```

doc_stanza = nlp_stanza(text)

stanza_entities = []

for sent in doc_stanza.sentences:
    for ent in sent.ents:
        stanza_entities.append({
            "Модель": "Stanza",
            "Текст сущности": ent.text,
            "Тип сущности": ent.type,
            "Начало": ent.start_char,
            "Конец": ent.end_char
        })

df_stanza = pd.DataFrame(stanza_entities)
df_stanza.head(20)

```

	Модель	Текст сущности	Тип сущности	Начало	Конец	
0	Stanza	СОДЕРЖАНИЕ	MISC	0	10	
1	Stanza	ПОСТАНОВКА ЗАДАЧИ.	PER	12	30	
2	Stanza	Описание объекта исследования	MISC	66	95	
3	Stanza	Предметная область	MISC	145	163	
4	Stanza	Постановка задачи	MISC	214	231	
5	Stanza	1 Цели	MISC	268	274	
6	Stanza	ОО	MISC	324	326	
7	Stanza	1 Цель	MISC	543	549	
8	Stanza	4.2 Задачи	MISC	576	586	
9	Stanza	Выводы о применимости результатов	MISC	696	729	
10	Stanza	7\nПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ	PER	758	785	
11	Stanza	Возможность использования алгоритма в низком поле	MISC	823	872	
12	Stanza	9\n8 Обучающая реализация алгоритма	MISC	891	925	
13	Stanza	Тл.	MISC	1045	1048	
14	Stanza	ЗАКЛЮЧЕНИЕ	LOC	1090	1100	
15	Stanza	Источники	MISC	1297	1306	
16	Stanza	ПОСТАНОВКА ЗАДАЧИ	MISC	1340	1357	
17	Stanza	MPT	MISC	1413	1416	
18	Stanza	К-пространство	MISC	1876	1890	
19	Stanza	Dx	MISC	2058	2060	

Далее:

New interactive sheet

Количество сущностей

df_stanza["Тип сущности"].value_counts()

count	
Тип сущности	
MISC	97
PER	25
ORG	18
LOC	10

dtype: int64

Объединение результатов двух моделей

```
df_all_entities = pd.concat([df_spacy, df_stanza], ignore_index=True)

df_all_entities.head(30)
```

	Модель	Текст сущности	Тип сущности	Начало	Конец	
0	SpaCy	Предметная область	LOC	145	163	
1	SpaCy	BP +++ оН ++ № 4+ ое\nDx\n\nDx	ORG	2033	2060	
2	SpaCy	РЕ	ORG	2119	2121	
3	SpaCy	Предметная область	LOC	2698	2716	
4	SpaCy	Цель	PER	3437	3441	
5	SpaCy	GRAPPA	ORG	3487	3493	
6	SpaCy	СКАРРА	ORG	3723	3729	
7	SpaCy	К-	LOC	4108	4110	
8	SpaCy	МРА	ORG	4198	4201	
9	SpaCy	GRAPPA	ORG	4514	4520	
10	SpaCy	К-	LOC	4760	4762	
11	SpaCy	Алгоритм	ORG	4947	4955	
12	SpaCy	ОКАРРА	ORG	4956	4962	
13	SpaCy	GRAPPA	ORG	5182	5188	
14	SpaCy	К-	LOC	6009	6011	
15	SpaCy	СКАРРА	ORG	6507	6513	
16	SpaCy	СВАРРА	ORG	7168	7174	
17	SpaCy	Frequency Encoding (ky)	ORG	7492	7515	
18	SpaCy	Амро	ORG	7518	7522	
19	SpaCy	К-	LOC	8688	8690	
20	SpaCy	Отарра	PER	8857	8863	
21	SpaCy	К-	LOC	8961	8963	
22	SpaCy	Епит	PER	9008	9012	
23	SpaCy	Низкое поле	LOC	9147	9158	
24	SpaCy	СКАРРА	ORG	9312	9318	
25	SpaCy	Ру@оп	ORG	9367	9372	
26	SpaCy	ИТМО	ORG	9382	9386	
27	SpaCy	НИР	ORG	9388	9391	
28	SpaCy	Original image Input image GRAPPA result\n\n\n...	ORG	9453	9549	
29	SpaCy	СКАРРА	ORG	9613	9619	

Далее: [New interactive sheet](#)

Сохранение результатов

```
df_spacy.to_csv("spacy_entities.csv", index=False, encoding="utf-8-sig")
df_stanza.to_csv("stanza_entities.csv", index=False, encoding="utf-8-sig")
df_all_entities.to_csv("all_ner_entities.csv", index=False, encoding="utf-8-sig")

print("Файлы сохранены:")
print("spacy_entities.csv")
print("stanza_entities.csv")
print("all_ner_entities.csv")
```

Файлы сохранены:
spacy_entities.csv
stanza_entities.csv
all_ner_entities.csv

Сравнительная таблица

```
spacy_counts = df_spacy["Тип сущности"].value_counts().reset_index()
spacy_counts.columns = ["Тип сущности", "SpaCy"]

stanza_counts = df_stanza["Тип сущности"].value_counts().reset_index()
stanza_counts.columns = ["Тип сущности", "Stanza"]

comparison_counts = pd.merge(
    spacy_counts,
    stanza_counts,
    on="Тип сущности",
    how="outer"
).fillna(0)

comparison_counts
```

	Тип сущности	SpaCy	Stanza
0	LOC	10.0	10
1	MISC	0.0	97
2	ORG	50.0	18
3	PER	8.0	25

Далее: [New interactive sheet](#)

Тип сущности	Что означает	SpaCy	Stanza
LOC	Локации, географические объекты	10	10
MISC	Прочие сущности	0	97
ORG	Организации, учреждения	50	18
PER	Персоны, имена людей	8	25

Какая из моделей за что приняла сущности:

```
# Примеры найденных сущностей по типам для SpaCy и Stanza

def show_entity_examples(df, model_name, n=10):
    print(f"\n===== {model_name}: примеры сущностей =====\n")

    for entity_type in sorted(df["Тип сущности"].unique()):
        examples = (
            df[df["Тип сущности"] == entity_type]["Текст сущности"]
            .dropna()
            .drop_duplicates()
            .head(n)
            .tolist()
        )

        print(f"{entity_type}:")
        for ex in examples:
            print(f"  - {ex}")
        print()

show_entity_examples(df_spacy, "SpaCy", n=10)
show_entity_examples(df_stanza, "Stanza", n=10)
```

- СОДЕРЖАНИЕ
- Описание объекта исследования
- Предметная область
- Постановка задачи
- 1 Цели
- 00
- 1 Цель
- 4.2 Задачи
- Выводы о применимости результатов
- Возможность использования алгоритма в низком поле

ORG:

- РЕ РЕ у

02 Dz!

- SHH HHHHHH HEE

00

- В СХЕМАТНУНОМ Вие
- ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ
- НИР
- ДЖЭТ ЛАБ
- ЗАКЛЮЧЕНИЕ
- David J Larkman D.J.
- Med. Biol.
- Wilson D.L.

PER:

- ПОСТАНОВКА ЗАДАЧИ.
- 7

ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ

- Отарра
- Епит
- Ваганов Ярослав Сергеевич
- Тымковский Константин Максимович
- Nana R.
- Hao T.
- Heberlein K.
- LaConte S.M.

Контекст для сущностей:

```
def get_context(text, start, end, window=80):
    left = max(0, start - window)
    right = min(len(text), end + window)

    context = text[left:right]
    entity = text[start:end]

    return context.replace(entity, f"[{entity}]")

def make_context_table(df, model_name, text, n=20):
    rows = []

    for _, row in df.head(n).iterrows():
        start = int(row["Начало"])
        end = int(row["Конец"])

        rows.append({
            "Модель": model_name,
            "Тип сущности": row["Тип сущности"],
            "Сущность": row["Текст сущности"],
            "Контекст": get_context(text, start, end)
        })

    return pd.DataFrame(rows)

context_spacy = make_context_table(df_spacy, "SpaCy", text, n=20)
context_stanza = make_context_table(df_stanza, "Stanza", text, n=20)

context_table = pd.concat([context_spacy, context_stanza], ignore_index=True)

context_table
```

Модель		Тип сущности	Сущность	Контекст
0	SpaCy	LOC	Предметная область	Описание объекта исследования..... ли...
1	SpaCy	ORG	BP +++ оН ++ № 4+ oe\nDx\n\nDx	\n\n90x 180y 180y ,---, 180y 180y 180:\n ...
2	SpaCy	ORG	PE	оН ++ № 4+ oe\nDx\n\nDx\n\nnox tnt trl deel de...
3	SpaCy	LOC	Предметная область	омощью алгоритмов.\n\n\n\n\nРисунок 2 — Пара: ...
4	SpaCy	PER	Цель	лученное путем ускорения, не\n\n\nотличалось от ...
5	SpaCy	ORG	GRAPPA	оренного».\n2.1 Цели и задачи\n\n\nЦель: теорети...
6	SpaCy	ORG	СКАРПА	, рассмотреть этапы автокалибровки и расчёта\n...
7	SpaCy	LOC	К-	компонент для сбора\n\n\nаппарата, но и с помощью ...
8	SpaCy	ORG	МРА	нства. Так как программная реализация таких ал...
9	SpaCy	ORG	GRAPPA	ического исследования\n\n\n4.1 Цель\n\n\nРазработа...
10	SpaCy	LOC	К-	, выделить этапы в\n\n\nпроцессе реконструкции, оп...
11	SpaCy	ORG	Алгоритм	модель алгоритма, разбитую на этапы.\n5 Резул...
12	SpaCy	ORG	ОКАРПА	лгоритма, разбитую на этапы.\n5 Результаты тео...
13	SpaCy	ORG	GRAPPA	мация о том, какая исследуемая область\n\n\nупавли...
14	SpaCy	LOC	К-	льсе) в\n\n\nчастотное пространство ([К-]пространс...
15	SpaCy	ORG	СКАРПА	ва с нескольких катушек)\n\n\nРисунок 4 — Получе...
16	SpaCy	ORG	СВАРПА	ft-f-8-t tt\n\n\nFrequency Encoding (k,...)\n\n\n\n...
17	SpaCy	ORG	Frequency Encoding (ky)	зон. Это же пространство изображено\n\n\nB CX€MaTH...
18	SpaCy	ORG	Амро	образено\n\n\nB CX€MaTHYHOМ Vue (Frequency Encodi...
19	SpaCy	LOC	К-	функция\n\n\nОКАРПА_гесоп(...), в качестве парамет...
20	Stanza	MISC	СОДЕРЖАНИЕ	[СОДЕРЖАНИЕ]\n\n\nПОСТАНОВКА ЗАДАЧИ.....
21	Stanza	PER	ПОСТАНОВКА ЗАДАЧИ.	СОДЕРЖАНИЕ\n\n\n[ПОСТАНОВКА ЗАДАЧИ.].....
22	Stanza	MISC	Описание объекта исследования	СОДЕРЖАНИЕ\n\n\nПОСТАНОВКА ЗАДАЧИ.....
23	Stanza	MISC	Предметная область	Описание объекта исследования..... ли...
24	Stanza	MISC	Постановка задачи	ния 2\n\n\n1.1 Предметная область..... ...
25	Stanza	MISC	1 Цели	ини иининининининининия 3\n\n\n2 Постановка задачи.....
26	Stanza	MISC	ОО	. нии иинининининни 3\n\n\n2.1 Цели и задачи.....
27	Stanza	MISC	1 Цель	ине 5\n\n\n4 Цель и задачи теоретического исследов...
28	Stanza	MISC	4.2 Задачи	ого исследования..... инь иининини 5\n\n\n...
29	Stanza	MISC	Выводы о применимости результатов	\n5 Результаты теоретического исследования.....
30	Stanza	PER	7\n\nПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ	инининининия 5\n\n\n6 Выводы о применимости резул...
31	Stanza	MISC	Возможность использования алгоритма в низком поле	ииининининия 7\n\n\nПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ.....
32	Stanza	MISC	9\n\n8 Обучающая реализация алгоритма	инининни 8\n\n\n7 Возможность использования алгорит...
33	Stanza	MISC	Тл.	ньне 9\n\n\n9 Реализация алгоритма как функция с...
34	Stanza	LOC	ЗАКЛЮЧЕНИЕ	рвиса реконструкции\n\n\n\n\nтомографа 1,5 Тл.....
35	Stanza	MISC	Источники	ссееcccсеccsscccccccescececccasccccscensec...
36	Stanza	MISC	ПОСТАНОВКА ЗАДАЧИ	secccesceesceaeaeaeessensecs 11\n\n\n1] Источни...
37	Stanza	MISC	МРТ	ь 11\n\n\n\nПОСТАНОВКА ЗАДАЧИ\n\n\n1 Описание объе...
38	Stanza	MISC	К-пространство	льности и многократно повторяют, получается на...

Далее: [New interactive sheet](#)

Далее: [New interactive sheet](#)

Пункт 5. Сравнение результатов SpaCy и Stanza

```
# Приводим сущности к единому формату для сравнения

df_spacy_compare = df_spacy.copy()
df_stanza_compare = df_stanza.copy()

df_spacy_compare["Текст сущности"] = df_spacy_compare["Текст сущности"].str.strip()
df_stanza_compare["Текст сущности"] = df_stanza_compare["Текст сущности"].str.strip()

df_spacy_compare["Текст сущности нижний"] = df_spacy_compare["Текст сущности"].str.lower()
df_stanza_compare["Текст сущности нижний"] = df_stanza_compare["Текст сущности"].str.lower()
```

```
common_entities = pd.merge(
    df_spacy_compare,
    df_stanza_compare,
    on="Текст сущности нижний",
    how="inner",
    suffixes=("_SpaCy", "_Stanza")
)

common_entities[[
    "Текст сущности_SpaCy",
    "Тип сущности_SpaCy",
    "Тип сущности_Stanza"
]].drop_duplicates().head(30)
```

	Текст сущности_SpaCy	Тип сущности_SpaCy	Тип сущности_Stanza	
0	Предметная область	LOC	MISC	
1	Предметная область	LOC	LOC	
4	GRAPPA	ORG	MISC	
11	СКAPPA	ORG	MISC	
19	MPA	ORG	MISC	
27	OKAPPA	ORG	MISC	
43	CBAPPA	ORG	MISC	
44	Амро	ORG	MISC	
45	Отарра	PER	PER	
46	Епит	PER	PER	
55	Py@оп	ORG	MISC	
56	ИТМО	ORG	LOC	
58	НИР	ORG	ORG	
67	РуФоп	PER	MISC	
68	России	LOC	LOC	
77	ЛаПа	ORG	MISC	
78	ДЖЭТ ЛАБ	ORG	ORG	
95	Нao T.	ORG	PER	
96	LaConte S.M.	ORG	PER	
97	Magnetic Resonance in Medicine	ORG	MISC	
99	Jakob P.M.	ORG	PER	
100	Heidemann R.M.	ORG	PER	
101	Nittka M.	ORG	PER	
102	Jellus V.	ORG	PER	
103	Wang J.	ORG	PER	
105	Kiefer B.	ORG	PER	
106	Wilson D.L.	ORG	ORG	
107	Huo D. Robust GRAPPA	ORG	PER	
110	ЗЕМЪЕ	ORG	LOC	
111	Nagarsekar G.	ORG	PER	

Сущности, найденные только SpaCy

```
spacy_only = df_spacy_compare[
    ~df_spacy_compare["Текст сущности нижний"].isin(
        df_stanza_compare["Текст сущности нижний"]
    )
]

spacy_only[[
    "Текст сущности",
    "Тип сущности",
    "Начало",
    "Конец"
]].drop_duplicates().head(30)
```

	Текст сущности	Тип сущности	Начало	Конец	
1	BP +++ oH ++ № 4+ oe\nDx\n\nDx	ORG	2033	2060	
2		PE	ORG	2119	2121
4		Цель	PER	3437	3441
7		К-	LOC	4108	4110
10		К-	LOC	4760	4762
11		Алгоритм	ORG	4947	4955
14		К-	LOC	6009	6011
17	Frequency Encoding (ky)	ORG	7492	7515	
19		К-	LOC	8688	8690
21		К-	LOC	8961	8963
23		Низкое поле	LOC	9147	9158
28	Original image Input image GRAPPA result\n\n\n...	ORG	9453	9549	
35		К-	LOC	10756	10758
38		Ваганов	PER	11413	11420
39		Ярослав Сергеевич	PER	11421	11438
40		ИСУ	ORG	11440	11443
41		Ру	PER	11496	11498
42		Константин Максимович	PER	11515	11536
43		ИСУ	ORG	11538	11541
46	Hu X. Cross-validation-\nbased kernel support ...	ORG	12336	12425	
48	Nunes R.G. Parallel magnetic resonance\nimaging	ORG	12504	12550	
55	Haase A. Generalized autocalibrating partially	ORG	12677	12723	
56		Magn Reson Med	ORG	12756	12770
59		United States Patent No	ORG	12833	12856
61	Detre J.A. Improved data reconstruction method...	ORG	12912	12969	
62		ДЖЭТ-ЛАБ	ORG	13030	13038

Сущности, найденные только Stanza

```
stanza_only = df_stanza_compare[
    ~df_stanza_compare["Текст сущности нижний"].isin(
        df_spacy_compare["Текст сущности нижний"]
    )
]

stanza_only[[
    "Текст сущности",
    "Тип сущности",
    "Начало",
    "Конец"
]].drop_duplicates().head(30)
```

```
"Конец"
]].drop_duplicates().head(30)
```

	Текст сущности	Тип сущности	Начало	Конец	
0	СОДЕРЖАНИЕ	MISC	0	10	
1	ПОСТАНОВКА ЗАДАЧИ.	PER	12	30	
2	Описание объекта исследования	MISC	66	95	
4	Постановка задачи	MISC	214	231	
5	1 Цели	MISC	268	274	
6	ОО	MISC	324	326	
7	1 Цель	MISC	543	549	
8	4.2 Задачи	MISC	576	586	
9	Выводы о применимости результатов	MISC	696	729	
10	7\nПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ	PER	758	785	
11	Возможность использования алгоритма в низком поле	MISC	823	872	
12	9\n8 Обучающая реализация алгоритма	MISC	891	925	
13	Тл.	MISC	1045	1048	
14	ЗАКЛЮЧЕНИЕ	LOC	1090	1100	
15	Источники	MISC	1297	1306	
16	ПОСТАНОВКА ЗАДАЧИ	MISC	1340	1357	
17	MPT	MISC	1413	1416	
18	K-пространство	MISC	1876	1890	
19	Dx	MISC	2058	2060	
20	Dy Dy\nG A	MISC	2088	2099	
21	PE PE y\n02 Dz!	ORG	2119	2133	
22	Gz	MISC	2139	2141	
23	aot Sty Navigator	MISC	2154	2171	
24	Методика получения MPT	MISC	2192	2214	
25	Пара: K-пространство	MISC	2653	2673	
28	AC3	MISC	2828	2831	
29	MP	MISC	3000	3002	
30	1 Цели	MISC	3420	3426	
34	M оп-О15с\розиге Азтеетеп@	MISC	4203	4228	
35	ТЕОРЕТИЧЕСКОЕ ИССЛЕДОВАНИЕ\n4	MISC	4385	4413	

Сравнение

```
comparison_summary = pd.DataFrame({
    "Показатель": [
        "Всего сущностей SpaCy",
        "Всего сущностей Stanza",
        "Общие сущности",
        "Только SpaCy",
        "Только Stanza"
    ],
    "Количество": [
        len(df_spacy_compare),
        len(df_stanza_compare),
        len(common_entities.drop_duplicates(subset=["Текст сущности нижний"])),
        len(spacy_only.drop_duplicates(subset=["Текст сущности нижний"])),
        len(stanza_only.drop_duplicates(subset=["Текст сущности нижний"]))
    ]
})

comparison_summary
```

	Показатель	Количество	
0	Всего сущностей SpaCy	68	
1	Всего сущностей Stanza	150	
2	Общие сущности	32	
3	Только SpaCy	20	
4	Только Stanza	93	

Далее: [New interactive sheet](#)

✓ Процент совпадений

```
spacy_unique = set(df_spacy_compare["Текст сущности нижний"])
stanza_unique = set(df_stanza_compare["Текст сущности нижний"])

intersection = spacy_unique & stanza_unique
union = spacy_unique | stanza_unique

jaccard_similarity = len(intersection) / len(union) if len(union) > 0 else 0

print(f"Количество уникальных сущностей SpaCy: {len(spacy_unique)}")
print(f"Количество уникальных сущностей Stanza: {len(stanza_unique)}")
print(f"Количество совпавших сущностей: {len(intersection)}")
print(f"Коэффициент сходства Жаккара: {jaccard_similarity:.3f}")
```

Количество уникальных сущностей SpaCy: 52
Количество уникальных сущностей Stanza: 125
Количество совпавших сущностей: 32
Коэффициент сходства Жаккара: 0.221

Коэффициент Жаккара показывает, насколько похожи наборы найденных сущностей.

Чем ближе значение к 1, тем сильнее совпадают модели. Чем ближе к 0, тем сильнее различаются результаты.

✓ Сравнение типов для одинаковых сущностей

```
type_comparison = common_entities[[
    "Текст сущности_SpaCy",
    "Тип сущности_SpaCy",
    "Тип сущности_Stanza"
]].drop_duplicates()

type_comparison["Тип совпал"] = (
    type_comparison["Тип сущности_SpaCy"] == type_comparison["Тип сущности_Stanza"]
)

type_comparison.head(30)
```

	Текст сущности_SpaCy	Тип сущности_SpaCy	Тип сущности_Stanza	Тип совпал
0	Предметная область	LOC	MISC	False
1	Предметная область	LOC	LOC	True
4	GRAPPA	ORG	MISC	False
11	СКАРПА	ORG	MISC	False
19	МРА	ORG	MISC	False
27	ОКАРПА	ORG	MISC	False
43	СВАРПА	ORG	MISC	False
44	Амро	ORG	MISC	False
45	Отарра	PER	PER	True
46	Епит	PER	PER	True
55	Рy@оп	ORG	MISC	False
56	ИТМО	ORG	LOC	False
58	НИР	ORG	ORG	True
67	РyФоп	PER	MISC	False
68	России	LOC	LOC	True
77	ЛаПа	ORG	MISC	False
78	ДЖЭТ ЛАБ	ORG	ORG	True
95	Нao T.	ORG	PER	False
96	LaConte S.M.	ORG	PER	False
97	Magnetic Resonance in Medicine	ORG	MISC	False
99	Jakob P.M.	ORG	PER	False
100	Heidemann R.M.	ORG	PER	False
101	Nittka M.	ORG	PER	False
102	Jellus V.	ORG	PER	False
103	Wang J.	ORG	PER	False
105	Kiefer B.	ORG	PER	False
106	Wilson D.L.	ORG	ORG	True
107	Huo D. Robust GRAPPA	ORG	PER	False
110	ЗЕМЪЕ	ORG	LOC	False
111	Nagarsekar G.	ORG	PER	False

Далее: [New interactive sheet](#)

Подсчёт совпадений и расхождений по типам

```
type_match_summary = type_comparison["Тип совпал"].value_counts().reset_index()
type_match_summary.columns = ["Совпадение типа", "Количество"]

type_match_summary
```

	Совпадение типа	Количество
0	False	24
1	True	9

Далее: [New interactive sheet](#)

```
comparison_summary.to_csv("ner_comparison_summary.csv", index=False, encoding="utf-8-sig")
common_entities.to_csv("ner_common_entities.csv", index=False, encoding="utf-8-sig")
spacy_only.to_csv("ner_spacy_only.csv", index=False, encoding="utf-8-sig")
stanza_only.to_csv("ner_stanza_only.csv", index=False, encoding="utf-8-sig")
```

```
type_comparison.to_csv("ner_type_comparison.csv", index=False, encoding="utf-8-sig")

print("Файлы сравнения сохранены.")

Файлы сравнения сохранены.
```

Пункт 6. Расчёт метрик качества NER-моделей

Метрика	Что показывает
Precision	Какая доля найденных моделью сущностей действительно правильная
Recall	Какую долю всех правильных сущностей модель смогла найти
F1-score	Среднее значение между Precision и Recall

```
sample_text = text[2000:2000+1500]

print(sample_text)
```

Dx

ox tnt trl deel dena

Dy | Dy
G A и | lh | 4
' "и "и PE PE y
02 Dz!

Gz ttt ttt th

aot Sty Navigator echoes

Рисунок 1 – Методика получения МРТ изображения, набор возбуждающих импульсов. Для уменьшения длительности исследования применяются методы параллельного сканирования (parallel imaging), в основе которых идет сбор не полной информации при использовании нескольких приемных катушек, информация с которых коррелирует и пространственные чувствительности которых различны. При этом недостающие данные «дособираются» искусственным способом – с помощью алгоритмов.

Рисунок 2 – Пара: K-пространство и МРТ изображение.

1.1 Предметная область

Одним из алгоритмов параллельного сканирования является SCAPPA. Алгоритм использует данные из автокалибровки (ACS, прескан) для вычисления весовых коэффициентов приемных катушек. По этим весам и

дописывается недостающая информация со сканирования.
2 Постановка задачи

Современные МР-томографы используют несколько приёмных катушек с разными пространственными чувствительностями, что позволяет восстанавливать изображения с неполным набором данных. Однако при реконструкции изображения с неполной выборкой могут возникнуть искажения и артефакты. Задача алгоритмов parallel imaging избежать дефектов при реконструкции, чтобы изображение, полученное путем ускорения, не отличалось от «неускоренного».

2.1 Цели и задачи

Цель: теоретически исследовать и описать алгоритм GRAPPA,
пост

Выбор эталонный сущностей

Фрагмент в тексте	Нормальный вид	Тип сущности	Почему
MPT	MPT	MISC	предметная область / технология
К-пространство	К-пространство	MISC	специальный термин
MPT изображение	MPT изображение	MISC	специальный термин
СКАРПА	GRAPPA	MISC	название алгоритма, OCR ошибся
АСЗ	ACS	MISC	автокалибровочные данные, OCR ошибся
MP-томографы	MP-томографы	MISC	технический объект
GRAPPA	GRAPPA	MISC	название алгоритма
Рисунок 1	Рисунок 1	MISC	элемент документа
Рисунок 2	Рисунок 2	MISC	элемент документа

```
gold_entities = [
    {"Текст сущности": "MPT", "Тип сущности": "MISC"},
    {"Текст сущности": "К-пространство", "Тип сущности": "MISC"},
    {"Текст сущности": "MPT изображение", "Тип сущности": "MISC"},
    {"Текст сущности": "GRAPPA", "Тип сущности": "MISC"},
    {"Текст сущности": "ACS", "Тип сущности": "MISC"},
    {"Текст сущности": "MP-томографы", "Тип сущности": "MISC"},
    {"Текст сущности": "Рисунок 1", "Тип сущности": "MISC"},
    {"Текст сущности": "Рисунок 2", "Тип сущности": "MISC"},
]

df_gold = pd.DataFrame(gold_entities)
df_gold
```

	Текст сущности	Тип сущности
0	MPT	MISC
1	К-пространство	MISC
2	MPT изображение	MISC
3	GRAPPA	MISC
4	ACS	MISC
5	MP-томографы	MISC
6	Рисунок 1	MISC
7	Рисунок 2	MISC

Далее: [New interactive sheet](#)

Подготовка для сравнения

```
def prepare_entities_for_metrics(df):
    prepared = df.copy()

    prepared["Текст сущности"] = (
        prepared["Текст сущности"]
        .astype(str)
        .str.strip()
        .str.lower()
    )

    prepared["Тип сущности"] = (
        prepared["Тип сущности"]
        .astype(str)
        .str.strip()
        .str.upper()
    )

    return prepared[["Текст сущности", "Тип сущности"]].drop_duplicates()

df_gold_prepared = prepare_entities_for_metrics(df_gold)
df_spacy_prepared = prepare_entities_for_metrics(df_spacy)
df_stanza_prepared = prepare_entities_for_metrics(df_stanza)
```

```
label_mapping = {
    "PERSON": "PER",
```

```
"PER": "PER",

"LOCATION": "LOC",
"LOC": "LOC",
"GPE": "LOC",

"ORGANIZATION": "ORG",
"ORG": "ORG",

"DATE": "DATE",

"WORK_OF_ART": "WORK_OF_ART",

"MISC": "MISC"
}

def normalize_labels(df):
    df = df.copy()
    df["Тип сущности"] = df["Тип сущности"].map(
        lambda x: label_mapping.get(x, x)
    )
    return df

df_gold_prepared = normalize_labels(df_gold_prepared)
df_spacy_prepared = normalize_labels(df_spacy_prepared)
df_stanza_prepared = normalize_labels(df_stanza_prepared)
```

Расчет

```
def calculate_entity_metrics(gold_df, pred_df):
    gold_set = set(
        zip(gold_df["Текст сущности"], gold_df["Тип сущности"])
    )

    pred_set = set(
        zip(pred_df["Текст сущности"], pred_df["Тип сущности"])
    )

    true_positive = len(gold_set & pred_set)
    false_positive = len(pred_set - gold_set)
    false_negative = len(gold_set - pred_set)

    precision = true_positive / (true_positive + false_positive) if (true_positive + false_positive) > 0 else 0
    recall = true_positive / (true_positive + false_negative) if (true_positive + false_negative) > 0 else 0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

    return {
        "True Positive": true_positive,
        "False Positive": false_positive,
        "False Negative": false_negative,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1
    }
```

```
spacy_metrics = calculate_entity_metrics(df_gold_prepared, df_spacy_prepared)
stanza_metrics = calculate_entity_metrics(df_gold_prepared, df_stanza_prepared)

metrics_table = pd.DataFrame([
    {"Модель": "SpaCy", **spacy_metrics},
    {"Модель": "Stanza", **stanza_metrics}
])

metrics_table
```

	Модель	True Positive	False Positive	False Negative	Precision	Recall	F1-score
0	SpaCy	0	52	8	0.000000	0.000	0.000000
1	Stanza	3	125	5	0.023438	0.375	0.044118

Далее: [New interactive sheet](#)

```
metrics_table_report = metrics_table.copy()

for col in ["Precision", "Recall", "F1-score"]:
    metrics_table_report[col] = metrics_table_report[col].round(3)

metrics_table_report
```

	Модель	True Positive	False Positive	False Negative	Precision	Recall	F1-score	
0	SpaCy	0	52	8	0.000	0.000	0.000	
1	Stanza	3	125	5	0.023	0.375	0.044	

Далее: [New interactive sheet](#)

```
def analyze_model_errors(gold_df, pred_df, model_name):
    gold_set = set(zip(gold_df["Текст сущности"], gold_df["Тип сущности"]))
    pred_set = set(zip(pred_df["Текст сущности"], pred_df["Тип сущности"]))

    correct = gold_set & pred_set
    extra = pred_set - gold_set
    missed = gold_set - pred_set

    rows = []

    for entity, label in correct:
        rows.append({
            "Модель": model_name,
            "Сущность": entity,
            "Тип": label,
            "Категория": "Верно найдена"
        })

    for entity, label in extra:
        rows.append({
            "Модель": model_name,
            "Сущность": entity,
            "Тип": label,
            "Категория": "Ложное срабатывание"
        })

    for entity, label in missed:
        rows.append({
            "Модель": model_name,
            "Сущность": entity,
            "Тип": label,
            "Категория": "Пропущена"
        })

    return pd.DataFrame(rows)

spacy_error_analysis = analyze_model_errors(
    df_gold_prepared,
    df_spacy_prepared,
    "SpaCy"
)

stanza_error_analysis = analyze_model_errors(
    df_gold_prepared,
    df_stanza_prepared,
    "Stanza"
)

error_analysis_table = pd.concat(
    [spacy_error_analysis, stanza_error_analysis],
    ignore_index=True
)

error_analysis_table.head(50)
```


Модель

Сущность

Тип

Категория

Далее:

0 SpaCy frequency encoding (ky) ORG Ложное срабатывание

1 SpaCy мpa ORG Ложное срабатывание

2 SpaCy nittka m. ORG Ложное срабатывание

3 SpaCy pe ORG Ложное срабатывание

4 SpaCy wang j. ORG Ложное срабатывание

5 SpaCy епит PER Ложное срабатывание

6 SpaCy huo d. robust grappa ORG Ложное срабатывание

7 SpaCy united states patent no ORG Ложное срабатывание

8 SpaCy delia improved data reconstruction method... ORG Ложное срабатывание

9 SpaCy heidemann g.m. ORG Ложное срабатывание

10 SpaCy py@op ORG Ложное срабатывание

11 SpaCy magn reson med ORG Ложное срабатывание

Вывод

Я рассмотрела только часть сущностей (эталонных, которые я ввела вручную), но если бы прочитала весь текст, то совпадений было бы больше.

Дообучение NER-модели SpaCy

Подготовка обучающего текста

```
train_text = ""
Рисунок 1 – Методика получения МРТ изображения, набор возбуждающих импульсов.
Для уменьшения длительности исследования применяются методы
параллельного сканирования, в основе которых идет сбор неполной информации
при использовании нескольких приемных катушек.

Рисунок 2 – Пара: К-пространство и МРТ изображение.

Одним из алгоритмов параллельного сканирования является SKAPPA.
Алгоритм использует данные из автокалибровки AC3, прескан, для
вычисления весовых коэффициентов приемных катушек.

Современные МР-томографы используют несколько приёмных катушек
с разными пространственными чувствительностями.

Цель: теоретически исследовать и описать алгоритм GRAPPA.
""
```

```
entities_to_find = [
    ("МРТ изображения", "MISC"),
    ("К-пространство", "MISC"),
    ("МРТ изображение", "MISC"),
    ("СКAPPA", "MISC"),
    ("AC3", "MISC"),
    ("MP-томографы", "MISC"),
    ("GRAPPA", "MISC"),
]
```

```
for entity_text, label in entities_to_find:
    start = train_text.find(entity_text)
    end = start + len(entity_text)

    print(entity_text, label, start, end)
```

13 SpaCy SKAPPA MISC 371 377

14 SpaCy AC3 MISC 424 427

15 SpaCy MP-томографы MISC 506 518

16 SpaCy GRAPPA MISC 656 662

17 SpaCy итмо ORG Ложное срабатывание

```
train_entities = []

for entity_text, label in entities_to_find:
    start = train_text.find(entity_text)
    end = start + len(entity_text)

    if start != -1:
        train_entities.append((start, end, label))

TRAIN_DATA = [
    (
        train_text,
```

```

        {
            "entities": train_entities
        }
    )
]

```

TRAIN_DATA

[('Рисунок 1 — Методика получения МРТ изображения, набор возбуждающих импульсов. Для уменьшения длительности исследования применяются методы параллельного сканирования, в основе которых идет сбор неполной информации при использовании нескольких приемных катушек. Рисунок 2 — Пара: К-пространство и МРТ изображение. Одним из алгоритмов параллельного сканирования является SKAPPA. Алгоритм использует данные из автокалибровки АСЗ, прескан, для вычисления весовых коэффициентов приемных катушек. Современные МР-томографы используют несколько приёмных катушек с разными пространственными чувствительностями. Цель: теоретически исследовать и описать алгоритм GRAPPA.',
 {'entities': [(32, 47, 'MISC'),
 (280, 294, 'MISC'),
 (297, 312, 'MISC'),
 (371, 377, 'MISC'),
 (424, 427, 'MISC'),
 (506, 518, 'MISC'),
 (656, 662, 'MISC')]}])]

✓ Проверка разметки

```

import spacy
from spacy.training import Example

nlp = spacy.load("ru_core_news_sm")

doc = nlp.make_doc(train_text)

example = Example.from_dict(
    doc,
    {
        "entities": train_entities
    }
)

print("Разметка корректна")

```

Разметка корректна

```

ner = nlp.get_pipe("ner")

for _, annotations in TRAIN_DATA:
    for start, end, label in annotations["entities"]:
        ner.add_label(label)

print("Метки добавлены")

```

Метки добавлены

✓ Дообучение модели

```

import random
from spacy.util import minibatch, compounding

optimizer = nlp.resume_training()

for epoch in range(30):
    random.shuffle(TRAIN_DATA)
    losses = {}

    batches = minibatch(TRAIN_DATA, size=compounding(1.0, 4.0, 1.5))

    for batch in batches:
        examples = []

        for text_item, annotations in batch:
            doc = nlp.make_doc(text_item)
            example = Example.from_dict(doc, annotations)
            examples.append(example)

        nlp.update(
            examples,

```

```

        drop=0.2,
        losses=losses,
        sgd=optimizer
    )

    print(f"Эпоха {epoch + 1}, потери: {losses}")

```

```

Эпоха 1, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(12.501884)}
Эпоха 2, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(12.884074)}
Эпоха 3, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(11.082508)}
Эпоха 4, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(10.409654)}
Эпоха 5, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(9.092919)}
Эпоха 6, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(8.698667)}
Эпоха 7, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(8.39642)}
Эпоха 8, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(8.23264)}
Эпоха 9, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(7.2158823)}
Эпоха 10, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(6.399421)}
Эпоха 11, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(5.722609)}
Эпоха 12, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(66.85577)}
Эпоха 13, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(55.88902)}
Эпоха 14, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(5.0827036)}
Эпоха 15, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(6.0117145)}
Эпоха 16, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(46.367355)}
Эпоха 17, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(56.543995)}
Эпоха 18, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(52.77779)}
Эпоха 19, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(58.965538)}
Эпоха 20, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(61.312515)}
Эпоха 21, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(37.873577)}
Эпоха 22, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(40.788506)}
Эпоха 23, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(36.59578)}
Эпоха 24, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(21.308228)}
Эпоха 25, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(21.278412)}
Эпоха 26, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(17.852007)}
Эпоха 27, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(15.083454)}
Эпоха 28, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(16.86806)}
Эпоха 29, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(13.022105)}
Эпоха 30, потери: {'tok2vec': 0.0, 'morphologizer': 0.0, 'parser': 0.0, 'ner': np.float32(9.22053)}

```

✓ Проверка после обучения

```

test_text = """
Алгоритм GRAPPA используется для восстановления МРТ изображения.
Для этого применяются данные АСЗ и информация из К-пространства.
Современные МР-томографы позволяют получать изображения с помощью нескольких катушек.
"""

```

```

doc_test = nlp(test_text)

for ent in doc_test.ents:
    print(ent.text, ent.label_)

```

```

GRAPPA MISC
МРТ изображения.
    MISC
АСЗ MISC
К-пространства. MISC
МР-томографы позволяют получать изображения с помощью нескольких катушек.
    MISC

```

```

fine_tuned_entities = []

for ent in doc_test.ents:
    fine_tuned_entities.append({
        "Текст сущности": ent.text,
        "Тип сущности": ent.label_,
        "Начало": ent.start_char,
        "Конец": ent.end_char
    })

df_fine_tuned = pd.DataFrame(fine_tuned_entities)
df_fine_tuned

```

	Текст сущности	Тип сущности	Начало	Конец	
0	GRAPPA	MISC	10	16	
1	MPT изображения.\n	MISC	49	66	
2	AC3	MISC	95	98	
3	К-пространства.	MISC	115	130	
4	MP-томографы позволяют получать изображения с ...	MISC	143	217	

Далее: [New interactive sheet](#)

```
output_dir = "spacy_finetuned_ner"
nlp.to_disk(output_dir)

print(f"Модель сохранена в папку: {output_dir}")
```

Модель сохранена в папку: spacy_finetuned_ner

```
nlp_finetuned = spacy.load("spacy_finetuned_ner")

doc_loaded = nlp_finetuned(test_text)

for ent in doc_loaded.ents:
    print(ent.text, ent.label_)

GRAPPA MISC
MPT изображения.
    MISC
AC3 MISC
К-пространства. MISC
MP-томографы позволяют получать изображения с помощью нескольких катушек.
    MISC
```

▼ Пункт 8. Сравнение модели до и после дообучения

Цель: показать, изменилась ли работа модели после fine-tuning: какие сущности она находила до обучения и какие стала находить после.

```
test_text = """
Алгоритм GRAPPA используется для восстановления MPT изображения.
Для этого применяются данные AC3 и информация из К-пространства.
Современные MP-томографы позволяют получать изображения с помощью нескольких приемных катушек.
Методы параллельного сканирования позволяют уменьшить длительность исследования.
"""
```

Базовая

```
nlp_base = spacy.load("ru_core_news_sm")

doc_base = nlp_base(test_text)

base_entities = []

for ent in doc_base.ents:
    base_entities.append({
        "Модель": "Базовая SpaCy",
        "Текст сущности": ent.text,
        "Тип сущности": ent.label_,
        "Начало": ent.start_char,
        "Конец": ent.end_char
    })

df_base_entities = pd.DataFrame(base_entities)
df_base_entities
```

	Модель	Текст сущности	Тип сущности	Начало	Конец	
0	Базовая SpaCy	AC3	ORG	95	98	

Дообученная

```
nlp_finetuned = spacy.load("spacy_finetuned_ner")

doc_finetuned = nlp_finetuned(test_text)

finetuned_entities = []

for ent in doc_finetuned.ents:
    finetuned_entities.append({
        "Модель": "Дообученная SpaCy",
        "Текст сущности": ent.text,
        "Тип сущности": ent.label_,
        "Начало": ent.start_char,
        "Конец": ent.end_char
    })

df_finetuned_entities = pd.DataFrame(finetuned_entities)
df_finetuned_entities
```

	Модель	Текст сущности	Тип сущности	Начало	Конец	
0	Дообученная SpaCy	GRAPPA	MISC	10	16	
1	Дообученная SpaCy	MPT изображения.\n	MISC	49	66	
2	Дообученная SpaCy	AC3	MISC	95	98	
3	Дообученная SpaCy	К-пространства.	MISC	115	130	
4	Дообученная SpaCy	MP-томографы позволяют получать изображения с ...	MISC	143	226	

Далее: [New interactive sheet](#)

Сравнение

```
df_before_after = pd.concat(
    [df_base_entities, df_finetuned_entities],
    ignore_index=True
)

df_before_after
```

	Модель	Текст сущности	Тип сущности	Начало	Конец	
0	Базовая SpaCy	AC3	ORG	95	98	
1	Дообученная SpaCy	GRAPPA	MISC	10	16	
2	Дообученная SpaCy	MPT изображения.\n	MISC	49	66	
3	Дообученная SpaCy	AC3	MISC	95	98	
4	Дообученная SpaCy	К-пространства.	MISC	115	130	
5	Дообученная SpaCy	MP-томографы позволяют получать изображения с ...	MISC	143	226	

Далее: [New interactive sheet](#)

```
before_after_counts = df_before_after.groupby(
    ["Модель", "Тип сущности"]
).size().reset_index(name="Количество")

before_after_counts
```

	Модель	Тип сущности	Количество	
0	Базовая SpaCy	ORG	1	
1	Дообученная SpaCy	MISC	5	

Далее: [New interactive sheet](#)

```
base_set = set(df_base_entities["Текст сущности"].str.lower()) if len(df_base_entities) > 0 else set()
finetuned_set = set(df_finetuned_entities["Текст сущности"].str.lower()) if len(df_finetuned_entities) > 0 else set()

only_finetuned = finetuned_set - base_set
only_base = base_set - finetuned_set
common = base_set & finetuned_set

comparison_result = pd.DataFrame({
    "Показатель": [
        "Найдены обеими моделями",
        "Найдены только базовой моделью",
        "Найдены только дообученной моделью"
    ],
    "Сущности": [
        ", ".join(common) if common else "-",
        ", ".join(only_base) if only_base else "-",
        ", ".join(only_finetuned) if only_finetuned else "-"
    ],
    "Количество": [
        len(common),
        len(only_base),
        len(only_finetuned)
    ]
})

comparison_result
```

	Показатель	Сущности	Количество
0	Найдены обеими моделями	асз	1
1	Найдены только базовой моделью	—	0
2	Найдены только дообученной моделью	мрт изображения.\n, мр-томографы позволяют пол...	4

Далее: [New interactive sheet](#)

Метрики базовой модели и дообученной

```
gold_entities_test = [
    {"Текст сущности": "GRAPPA", "Тип сущности": "MISC"},
    {"Текст сущности": "МРТ изображения", "Тип сущности": "MISC"},
    {"Текст сущности": "АСЗ", "Тип сущности": "MISC"},
    {"Текст сущности": "К-пространства", "Тип сущности": "MISC"},
    {"Текст сущности": "МР-томографы", "Тип сущности": "MISC"},
]

df_gold_test = pd.DataFrame(gold_entities_test)
df_gold_test
```

	Текст сущности	Тип сущности
0	GRAPPA	MISC
1	МРТ изображения	MISC
2	АСЗ	MISC
3	К-пространства	MISC
4	МР-томографы	MISC

Далее: [New interactive sheet](#)

```
df_base_prepared = prepare_entities_for_metrics(df_base_entities)
df_finetuned_prepared = prepare_entities_for_metrics(df_finetuned_entities)
df_gold_test_prepared = prepare_entities_for_metrics(df_gold_test)

df_base_prepared = normalize_labels(df_base_prepared)
df_finetuned_prepared = normalize_labels(df_finetuned_prepared)
df_gold_test_prepared = normalize_labels(df_gold_test_prepared)
```

```
base_metrics = calculate_entity_metrics(
    df_gold_test_prepared,
```

```
df_base_prepared
)

finetuned_metrics = calculate_entity_metrics(
    df_gold_test_prepared,
    df_finetuned_prepared
)

before_after_metrics = pd.DataFrame([
    {"Модель": "Базовая SpaCy", **base_metrics},
    {"Модель": "Дообученная SpaCy", **finetuned_metrics}
])

for col in ["Precision", "Recall", "F1-score"]:
    before_after_metrics[col] = before_after_metrics[col].round(3)

before_after_metrics
```

	Модель	True Positive	False Positive	False Negative	Precision	Recall	F1-score	
0	Базовая SpaCy	0	1	5	0.0	0.0	0.0	
1	Дообученная SpaCy	2	3	3	0.4	0.4	0.4	

Далее: [New interactive sheet](#)

```
from spacy import displacy

displacy.render(doc_base, style="ent", jupyter=True)
```

Алгоритм GRAPPA используется для восстановления MPT изображения.

Для этого применяются данные AC3 ORG и информация из K-пространства.

Современные МР-томографы позволяют получать изображения с помощью нескольких приемных катушек.

Методы параллельного сканирования позволяют уменьшить длительность исследования.

```
displacy.render(doc_finetuned, style="ent", jupyter=True)
```

Алгоритм GRAPPA MISC используется для восстановления MPT изображения. MISC Для этого применяются данные AC3 MISC и информация из K-пространства. MISC

```
df_before_after.to_csv("ner_before_after_entities.csv", index=False, encoding="utf-8-sig")
before_after_counts.to_csv("ner_before_after_counts.csv", index=False, encoding="utf-8-sig")
comparison_result.to_csv("ner_before_after_comparison.csv", index=False, encoding="utf-8-sig")
before_after_metrics.to_csv("ner_before_after_metrics.csv", index=False, encoding="utf-8-sig")

print("Результаты сравнения до и после дообучения сохранены.")

Результаты сравнения до и после дообучения сохранены.
```

Пункт 9. Использование LLM для извлечения сущностей

Сравним два подхода:

Подход	Смысл
Zero-shot	Даём модели только инструкцию, без примеров
Few-shot	Даём модели инструкцию + несколько примеров правильной разметки

```
!pip install transformers accelerate bitsandbytes pandas -q
```

60.7/60.7 MB 7.4 MB/s eta 0:00:00

```
import pandas as pd
import torch
```



```
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
```

Zero shot

```
model_name = "Qwen/Qwen2.5-1.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

llm_pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%                               660/660 [00:00<00:00, 20.0kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
tokenizer_config.json:   7.30k/? [00:00<00:00, 170kB/s]

vocab.json:      2.78M/? [00:00<00:00, 53.0MB/s]

merges.txt:      1.67M/? [00:00<00:00, 42.6MB/s]

tokenizer.json:   7.03M/? [00:00<00:00, 86.4MB/s]
`torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100%                        3.09G/3.09G [00:43<00:00, 185MB/s]

Loading weights: 100%                          338/338 [00:16<00:00, 19.18it/s, Materializing param=model.norm.weight]

generation_config.json: 100%                    242/242 [00:00<00:00, 8.42kB/s]
```

```
llm_text = """
Рисунок 1 – Методика получения МРТ изображения, набор возбуждающих импульсов.
Для уменьшения длительности исследования применяются методы параллельного сканирования.

Рисунок 2 – Пара: K-пространство и МРТ изображение.

Одним из алгоритмов параллельного сканирования является GRAPPA.
Алгоритм использует данные из автокалибровки ACS для вычисления весовых коэффициентов приемных катушек.

Современные МР-томографы используют несколько приёмных катушек с разными пространственными чувствительностями.
"""
```

```
zero_shot_prompt = f"""
Ты выполняешь задачу извлечения именованных сущностей из русского технического текста.

Нужно найти сущности следующих типов:
- PERSON — имена людей;
- LOCATION — географические объекты;
- ORGANIZATION — организации и учреждения;
- DATE — даты и периоды;
- WORK_OF_ART — произведения искусства;
- MISC — специальные термины, алгоритмы, технические обозначения.

Верни результат строго в виде таблицы:
Сущность | Тип | Объяснение

Текст:
{llm_text}
"""
```

```

zero_shot_result = llm_pipe(
    zero_shot_prompt,
    max_new_tokens=500,
    do_sample=False
)

print(zero_shot_result[0]["generated_text"])

```

GRAPPA эффективно работает с данными, полученным из разных источников.

GRAPPA способна обрабатывать большое количество информации.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть применена к различным материалам.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAP

▼ Few shot

```
few_shot_prompt = f"""
Ты выполняешь задачу извлечения именованных сущностей из русского технического текста.
```

Типы сущностей:

- PERSON — имена людей;
- LOCATION — географические объекты;
- ORGANIZATION — организации и учреждения;
- DATE — даты и периоды;
- WORK_OF_ART — произведения искусства;
- MISC — специальные термины, алгоритмы, технические обозначения.

Пример 1:

Текст: Алгоритм GRAPPA используется для реконструкции МРТ изображения.

Ответ:

GRAPPA | MISC | название алгоритма

МРТ изображение | MISC | технический термин

Пример 2:

Текст: Данные ACS применяются при восстановлении К-пространства.

Ответ:

ACS | MISC | техническое обозначение автокалибровочных данных

К-пространство | MISC | термин из области МРТ

Теперь выполни такую же разметку для текста ниже.

Текст:

{llm_text}

Ответ:

""

```
few_shot_result = llm_pipe(
    few_shot_prompt,
    max_new_tokens=500,
    do_sample=False
)
```

```
print(few_shot_result[0]["generated_text"])
```

Вспомогательные данные на основе результатов для выполнения весовых коэффициентов приёмных катушек.

Современные МР-томографы используют несколько приёмных катушек с разными пространственными чувствительностями.

Ответ:

Рисунок 1 | MISC | техническая диаграмма

МРТ изображение | MISC | технический термин

К-пространство | MISC | термин из области МРТ

PARA | MISC | пара (математическая концепция)

GRAPPA | ORGANIZATION | алгоритм

ACS | MISC | техническое обозначение автокалибровочных данных

Параллельное сканирование | MISC | термин из области МРТ

Весовые коэффициенты | MISC | термин из области МРТ

МР-томографы | ORGANIZATION | оборудование

приёмные катушки | MISC | технические устройства

различные пространственные чувствительности | MISC | термин из области МРТ

```
```python
```

```
def extract_entities(text):
 entities = []
```

```
entity_type = "LOCATION"

elif word == "точка" or word ==
```

## ✓ Сохранение результатов

```
with open("llm_zero_shot_result.txt", "w", encoding="utf-8") as f:
 f.write(zero_shot_result[0]["generated_text"])

with open("llm_few_shot_result.txt", "w", encoding="utf-8") as f:
 f.write(few_shot_result[0]["generated_text"])

print("Результаты LLM сохранены.")
```

Результаты LLM сохранены.

## ✓ Пункт 10. Итоговое сравнение подходов

Цель этапа

Нужно сравнить, какой подход оказался удобнее и практичнее:

- готовые NER-модели;
- дообучение модели;
- LLM через zero-shot и few-shot prompting.

## ✓ Создание итоговой сравнительной таблицы

```
import pandas as pd

def safe_len(df):
 """Безопасно возвращает количество строк в DataFrame."""
 try:
 return len(df)
 except:
 return 0

def safe_unique_count(df, column):
 """Безопасно возвращает количество уникальных значений."""
 try:
 return df[column].dropna().nunique()
 except:
 return 0

def safe_entity_types(df):
 """Безопасно возвращает список типов сущностей."""
 try:
 return ", ".join(sorted(df["Тип сущности"].dropna().unique()))
 except:
 return "-"

def get_metric(metrics_df, model_name, metric_name):
 """Достаёт значение метрики из таблицы метрик."""
 try:
 value = metrics_df.loc[
 metrics_df["Модель"] == model_name,
 metric_name
].values[0]
 return round(float(value), 3)
 except:
 return None
```

```
spacy_total = safe_len(df_spacy)
stanza_total = safe_len(df_stanza)

spacy_unique = safe_unique_count(df_spacy, "Текст сущности")
stanza_unique = safe_unique_count(df_stanza, "Текст сущности")
```

```

spacy_types = safe_entity_types(df_spacy)
stanza_types = safe_entity_types(df_stanza)

print("SpaCy:")
print("Всего сущностей:", spacy_total)
print("Уникальных сущностей:", spacy_unique)
print("Типы сущностей:", spacy_types)

print("\nStanza:")
print("Всего сущностей:", stanza_total)
print("Уникальных сущностей:", stanza_unique)
print("Типы сущностей:", stanza_types)

```

```

SpaCy:
Всего сущностей: 68
Уникальных сущностей: 52
Типы сущностей: LOC, ORG, PER

Stanza:
Всего сущностей: 150
Уникальных сущностей: 127
Типы сущностей: LOC, MISC, ORG, PER

```

```

base_total = safe_len(df_base_entities)
finetuned_total = safe_len(df_finetuned_entities)

base_unique = safe_unique_count(df_base_entities, "Текст сущности")
finetuned_unique = safe_unique_count(df_finetuned_entities, "Текст сущности")

base_types = safe_entity_types(df_base_entities)
finetuned_types = safe_entity_types(df_finetuned_entities)

print("Базовая SpaCy:")
print("Всего сущностей:", base_total)
print("Уникальных сущностей:", base_unique)
print("Типы сущностей:", base_types)

print("\nДообученная SpaCy:")
print("Всего сущностей:", finetuned_total)
print("Уникальных сущностей:", finetuned_unique)
print("Типы сущностей:", finetuned_types)

```

```

Базовая SpaCy:
Всего сущностей: 1
Уникальных сущностей: 1
Типы сущностей: ORG

Дообученная SpaCy:
Всего сущностей: 5
Уникальных сущностей: 5
Типы сущностей: MISC

```

# Метрики для SpaCy и Stanza из этапа сравнения NER-моделей

```

spacy_precision = get_metric(metrics_table_report, "SpaCy", "Precision")
spacy_recall = get_metric(metrics_table_report, "SpaCy", "Recall")
spacy_f1 = get_metric(metrics_table_report, "SpaCy", "F1-score")

stanza_precision = get_metric(metrics_table_report, "Stanza", "Precision")
stanza_recall = get_metric(metrics_table_report, "Stanza", "Recall")
stanza_f1 = get_metric(metrics_table_report, "Stanza", "F1-score")

Метрики для базовой и дообученной SpaCy

base_precision = get_metric(before_after_metrics, "Базовая SpaCy", "Precision")
base_recall = get_metric(before_after_metrics, "Базовая SpaCy", "Recall")
base_f1 = get_metric(before_after_metrics, "Базовая SpaCy", "F1-score")

finetuned_precision = get_metric(before_after_metrics, "Дообученная SpaCy", "Precision")
finetuned_recall = get_metric(before_after_metrics, "Дообученная SpaCy", "Recall")
finetuned_f1 = get_metric(before_after_metrics, "Дообученная SpaCy", "F1-score")

```

```

def get_llm_output(result):
 try:
 return result[0]["generated_text"]
 except Exception:

```

```

return ""

zero_shot_text = get_llm_output(zero_shot_result) if "zero_shot_result" in globals() else ""
few_shot_text = get_llm_output(few_shot_result) if "few_shot_result" in globals() else ""

print("Zero-shot output:")
print(zero_shot_text[-1500:])

print("\nFew-shot output:")
print(few_shot_text[-1500:])

```

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAPPA может быть адаптирована под различные условия использования.

GRAP

Few-shot output:

тельности | MISC | термин из области МРТ

```
```python
```

```

def extract_entities(text):
    entities = []

    # Split the text into sentences for easier processing
    sentences = text.split('. ')

    for sentence in sentences:
        if 'и' in sentence or 'или' in sentence:
            continue

        words = sentence.lower().split()

        for i in range(len(words)):
            word = words[i]

            if word.isupper() and len(word) > 3:
                entity_type = "MISC"

                if word == "анализ":
                    entity_type = "WORK_OF_ART"

                elif word == "восстановление" or word == "реализация":
                    entity_type = "ORGANIZATION"

                elif word == "изображение" or word == "изображений":
                    entity_type = "WORK_OF_ART"

                elif word == "автокалибровочные" or word == "автокалибровочная":
                    entity_type = "MISC"

                elif word == "радиоактивный" or word == "радиоактивные":
                    entity_type = "PERSON"

                elif word == "среда" or word == "среды":
                    entity_type = "LOCATION"

                elif word == "объект" or word == "объектов":
                    entity_type = "LOCATION"

                elif word == "точка" or word ==

```

```

import re
import pandas as pd

```

```

allowed_labels = {
    "PERSON", "PER",
    "LOCATION", "LOC",
    "ORGANIZATION", "ORG",
    "DATE",
    "WORK_OF_ART",
    "MISC"
}

```

```

label_mapping = {
    "PERSON": "PER",
    "PER": "PER",
    "LOCATION": "LOC",
    "LOC": "LOC",
    "ORGANIZATION": "ORG",
    "ORG": "ORG",
    "DATE": "DATE",
    "WORK_OF_ART": "WORK_OF_ART",
    "MISC": "MISC"
}

def parse_llm_entities(output_text, approach_name):
    rows = []

    for line in output_text.splitlines():
        line = line.strip()

        if "|" not in line:
            continue

        parts = [p.strip() for p in line.split("|")]

        if len(parts) < 2:
            continue

        entity = parts[0]
        label = parts[1].upper()

        if entity.lower() in ["сущность", "entity", "ответ", "текст"]:
            continue

        if label not in allowed_labels:
            continue

        rows.append({
            "Подход": approach_name,
            "Текст сущности": entity,
            "Тип сущности": label_mapping.get(label, label),
            "Объяснение": parts[2] if len(parts) > 2 else ""
        })

    return pd.DataFrame(rows)

df_zero_shot_llm = parse_llm_entities(zero_shot_text, "LLM zero-shot")
df_few_shot_llm = parse_llm_entities(few_shot_text, "LLM few-shot")

llm_results_table = pd.concat(
    [df_zero_shot_llm, df_few_shot_llm],
    ignore_index=True
)

llm_results_table

```



	Подход	Текст сущности	Тип сущности	Объяснение
0	LLM few-shot	GRAPPA	MISC	название алгоритма
1	LLM few-shot	MPT изображение	MISC	технический термин
2	LLM few-shot	ACS	MISC	техническое обозначение автокалибровочных данных
3	LLM few-shot	К-пространство	MISC	термин из области MPT
4	LLM few-shot	Рисунок 1	MISC	техническая диаграмма
5	LLM few-shot	MPT изображение	MISC	технический термин
6	LLM few-shot	К-пространство	MISC	термин из области MPT
7	LLM few-shot	PARA	MISC	пара (математическая концепция)
8	LLM few-shot	GRAPPA	ORG	алгоритм
9	LLM few-shot	ACS	MISC	техническое обозначение автокалибровочных данных
10	LLM few-shot	Параллельное сканирование	MISC	термин из области MPT
11	LLM few-shot	Весовые коэффициенты	MISC	термин из области MPT
12	LLM few-shot	МР-томографы	ORG	оборудование
13	LLM few-shot	приёмные катушки	MISC	технические устройства
14	LLM few-shot	различные пространственные чувствительности	MISC	термин из области MPT

Далее: [New interactive sheet](#)

```
print("df_zero_shot_llm columns:", df_zero_shot_llm.columns.tolist())
print("df_few_shot_llm columns:", df_few_shot_llm.columns.tolist())

display(df_zero_shot_llm.head())
display(df_few_shot_llm.head())
```

```
df_zero_shot_llm columns: []
df_few_shot_llm columns: ['Подход', 'Текст сущности', 'Тип сущности', 'Объяснение']
```



	Подход	Текст сущности	Тип сущности	Объяснение
0	LLM few-shot	GRAPPA	MISC	название алгоритма
1	LLM few-shot	MPT изображение	MISC	технический термин
2	LLM few-shot	ACS	MISC	техническое обозначение автокалибровочных данных
3	LLM few-shot	К-пространство	MISC	термин из области MPT
4	LLM few-shot	Рисунок 1	MISC	техническая диаграмма

```
def prepare_entities_for_metrics_auto(df):
    prepared = df.copy()

    # Если таблица пустая и без колонок — создаём нужные колонки
    if "Текст сущности" not in prepared.columns:
        prepared["Текст сущности"] = ""

    if "Тип сущности" not in prepared.columns:
        prepared["Тип сущности"] = ""

    prepared["Текст сущности"] = (
        prepared["Текст сущности"]
        .astype(str)
        .str.strip()
        .str.lower()
    )

    prepared["Тип сущности"] = (
        prepared["Тип сущности"]
        .astype(str)
        .str.strip()
        .str.upper()
    )

    prepared["Тип сущности"] = prepared["Тип сущности"].map(
```



```
        lambda x: label_mapping.get(x, x)
    )

    prepared = prepared[
        (prepared["Текст сущности"] != "") &
        (prepared["Тип сущности"] != "")
    ]

    return prepared[["Текст сущности", "Тип сущности"]].drop_duplicates()
```

```
def calculate_entity_metrics_auto(gold_df, pred_df):
    gold_set = set(zip(gold_df["Текст сущности"], gold_df["Тип сущности"]))
    pred_set = set(zip(pred_df["Текст сущности"], pred_df["Тип сущности"]))

    tp = len(gold_set & pred_set)
    fp = len(pred_set - gold_set)
    fn = len(gold_set - pred_set)

    precision = tp / (tp + fp) if (tp + fp) else 0
    recall = tp / (tp + fn) if (tp + fn) else 0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0

    return {
        "True Positive": tp,
        "False Positive": fp,
        "False Negative": fn,
        "Precision": round(precision, 3),
        "Recall": round(recall, 3),
        "F1-score": round(f1, 3)
    }

# Берём эталонную разметку автоматически из уже существующих переменных
if "df_gold_test" in globals():
    gold_for_llm = df_gold_test
elif "df_gold" in globals():
    gold_for_llm = df_gold
else:
    gold_for_llm = pd.DataFrame(columns=["Текст сущности", "Тип сущности"])

gold_for_llm_prepared = prepare_entities_for_metrics_auto(gold_for_llm)

zero_llm_prepared = prepare_entities_for_metrics_auto(df_zero_shot_llm)
few_llm_prepared = prepare_entities_for_metrics_auto(df_few_shot_llm)

zero_shot_metrics = calculate_entity_metrics_auto(
    gold_for_llm_prepared,
    zero_llm_prepared
)

few_shot_metrics = calculate_entity_metrics_auto(
    gold_for_llm_prepared,
    few_llm_prepared
)

llm_metrics_table = pd.DataFrame([
    {"Подход": "LLM zero-shot", **zero_shot_metrics},
    {"Подход": "LLM few-shot", **few_shot_metrics}
])

llm_metrics_table
```

	Подход	True Positive	False Positive	False Negative	Precision	Recall	F1-score	
0	LLM zero-shot	0	0	5	0.000	0.0	0.000	
1	LLM few-shot	1	11	4	0.083	0.2	0.118	

Далее: [New interactive sheet](#)

```
def calculate_llm_accuracy(df, approach_name):
    try:
        subset = df[df["Подход"] == approach_name]

        if len(subset) == 0:
```